# DeepSeek V4.1 Flash Engram

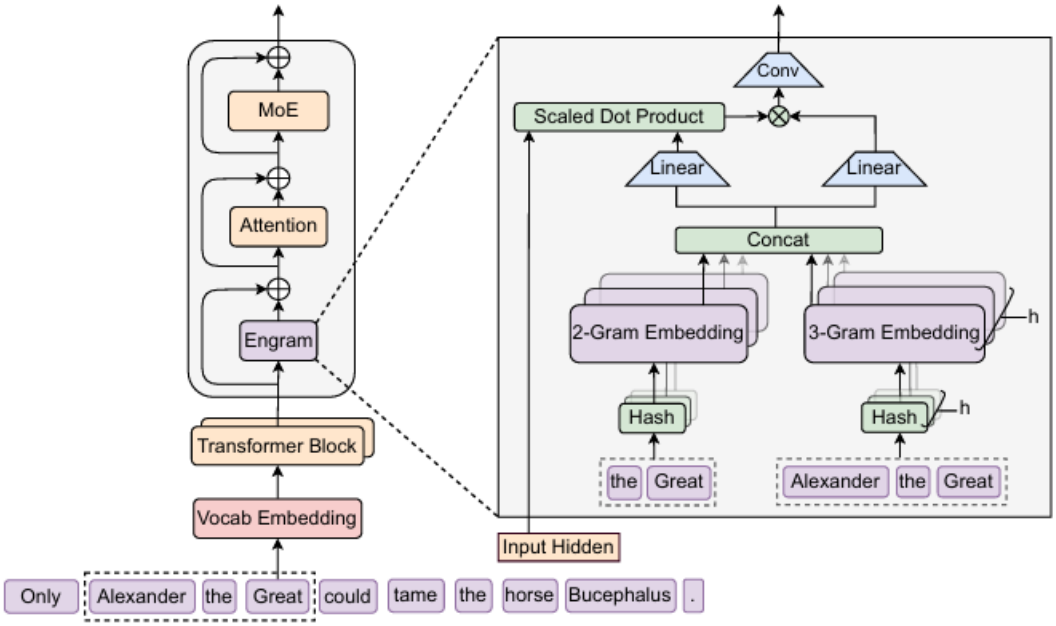



Figure 1: Engram 架构细节图。左侧展示Engram 在Transformer Block 中的位置：输入经Vocab Embedding 和前序Transformer Block 后进入当前块，当前块内部顺序为Engram → Attention → MoE。右侧展开Engram 内部核心组件：对当前位置构造$n$-gram 后缀（实际配置$n = 2, 3, 4$，图中示意2-Gram/3-Gram），经Hash 映射查表、Concat 拼接、Linear 投影得到K/V；以当前隐藏状态为Q，与K 做Scaled Dot Product 门控，再作用于V，最后经Conv1D + SiLU 与残差注入回隐藏状态。

## 1 真实配置

| 项目 | 真实V4.1-Flash |
|---|---|
| 压缩词表 | 99,092 |
| 后缀n-gram | $n = 2, 3, 4$ |
| 哈希头数 | 8 |
| head_dim | 256 |
| 每行存储 | 256 字节FP8 |
| Engram 层 | 2 层 |
| 表行数 | 每层约3.84 亿行 |

真实中，每个token 位置在每个Engram 层取：3 个n-gram 阶数×8 个哈希头= 24 行，每行256 字节FP8，拼成6144 字节记忆向量。

## 2 输入与压缩

原始token：`["The", "capital", "of", "France", "is"]`。压缩映射：The→the(1)，capital→capital(2)，of→of(3)，压缩ID 序列为`[1, 2, 3, 4, 5]`，当前位置是最后一个token `is`，Engram 在此介入。

# 3 构造后缀n-gram

在`is` 位置取最近2、3、4 个压缩token：2-gram `(4,5)`，3-gram `(3,4,5)`，4-gram `(2,3,4,5)`。

# 4 多头部哈希与检索

真实中，每个$(n, head)$ 对分配一个素数$p$ 和偏移$off$，保证桶区间互不重叠。哈希公式：

$$h = (\mathrm{id}_1 \cdot m_1 \oplus \mathrm{id}_2 \cdot m_2 \oplus \cdots) \bmod p, \qquad \text{bucket} = off + h.$$

以下用6 个$(n, head)$ 对作为计算示意；真实中每层共有$3 \times 8 = 24$ 个$(n, head)$ 对。

```
(2,0) n-gram (4,5):  4*31=124, 5*37=185, XOR=197, mod101=96, off0 -> bucket 96
(2,1) n-gram (4,5):  4*41=164, 5*43=215, XOR=115, mod103=12, off101 -> bucket 113
(3,0) n-gram (3,4,5): 3*17=51, 4*19=76, 5*23=115, XOR=12, mod107=12, off204 ->
    bucket 216
(3,1) n-gram (3,4,5): 3*29=87, 4*31=124, 5*37=185, XOR=146, mod109=37, off311 ->
    bucket 348
(4,0) n-gram (2,3,4,5): 2*13=26, 3*17=51, 4*19=76, 5*23=115, XOR=22, mod113=22,
    off420 -> bucket 442
(4,1) n-gram (2,3,4,5): 2*11=22, 3*13=39, 4*17=68, 5*19=95, XOR=42, mod127=42,
    off533 -> bucket 575
```

命中的6 个桶为`96, 113, 216, 348, 442, 575`。真实中每行是256 维FP8；下表只展示每行的前4 维作为计算示意，其余252 维同理。

| 桶索引 | 行向量（仅展示前4 维） |
|---|---|
| 96 | [0.90, 0.10, 0.80, 0.20, . . . ] |
| 113 | [0.70, 0.00, 0.60, 0.10, . . . ] |
| 216 | [0.80, 0.20, 0.90, 0.30, . . . ] |
| 348 | [0.60, 0.10, 0.70, 0.20, . . . ] |
| 442 | [0.95, 0.00, 0.85, 0.10, . . . ] |
| 575 | [0.50, 0.30, 0.40, 0.20, . . . ] |

真实中，6 个桶只是示意；实际是24 行，每行256 维，拼接得到：

$$m \in \mathbb{R}^{6144}, \qquad m = [m_1; m_2; \ldots; m_{24}], \quad m_j \in \mathbb{R}^{256}.$$

若只写前4 维示意，则为：

```
m[0:4] = [0.90, 0.10, 0.80, 0.20]
m[4:8] = [0.70, 0.00, 0.60, 0.10]
...
```

真实中，这里就是24 行× 256 字节= 6144 字节。

# 5 投影成K 和V

Engram 借用Attention 的QKV 思想：$Q$ 是当前隐藏状态$h$（含上下文），$K$ 由$m$ 投影得到（表示“记忆在说什么”），$V$ 由$m$ 投影得到（表示“如果采纳，注入什么”）。

真实中，$m \in \mathbb{R}^{6144}$，投影矩阵为：

$$W_V \in \mathbb{R}^{256\times 6144}, \qquad W_{K1}, W_{K2} \in \mathbb{R}^{256\times 6144}.$$

因此：

$$V = W_V m \in \mathbb{R}^{256}, \qquad K_1 = W_{K1} m \in \mathbb{R}^{256}, \qquad K_2 = W_{K2} m \in \mathbb{R}^{256}.$$

这里$K_1$ 和$K_2$ 不是两份不同的记忆，而是**同一份检索记忆**$m$ **经过两个不同Key 投影矩阵得到的两个"记忆摘要"**，分别用来和两个分支隐藏状态$h_1, h_2$ 做匹配；Value 是两个分支共享的。

微缩数值演示只取前6 个桶、每个桶前4 维，用对角块权重模拟$W_V, W_{K1}, W_{K2}$ 的局部块；真实计算对全部24 个桶、全部256 维进行。这里'row1' 至'row6' 分别代表对应桶向量截取的前4 维。真实的$W_{K1}$ 是一个$256 \times 6144$ 的矩阵，这里用对角块权重模拟了它在第0~3 个输出维度的局部计算。

共享Value 投影权重（6 个桶，每行和为1）：

$$w_V = [0.10,\ 0.15,\ 0.20,\ 0.20,\ 0.20,\ 0.15]$$

分支Key 投影权重：

$$w_{K1} = [0.40,\ 0.35,\ 0.25,\ 0,\ 0,\ 0]$$

$$w_{K2} = [0,\ 0,\ 0,\ 0.25,\ 0.35,\ 0.40]$$

以下只展示前4 个输出维：

```
V[0:4]:
  V[0] = 0.10*0.90 + 0.15*0.70 + 0.20*0.80 + 0.20*0.60 + 0.20*0.95 + 0.15*0.50 =
      0.740
  V[1] = 0.10*0.10 + 0.15*0.00 + 0.20*0.20 + 0.20*0.10 + 0.20*0.00 + 0.15*0.30 =
      0.115
  V[2] = 0.10*0.80 + 0.15*0.60 + 0.20*0.90 + 0.20*0.70 + 0.20*0.85 + 0.15*0.40 =
      0.720
  V[3] = 0.10*0.20 + 0.15*0.10 + 0.20*0.30 + 0.20*0.20 + 0.20*0.10 + 0.15*0.20 =
      0.185

K1[0:4] = 0.40*row1 + 0.35*row2 + 0.25*row3
        = [0.805, 0.090, 0.755, 0.190]

K2[0:4] = 0.25*row4 + 0.35*row5 + 0.40*row6
        = [0.683, 0.145, 0.633, 0.165]
```

真实中$V, K_1, K_2$ 都是256 维；上面只展开前4 维，其余252 维按同样方式计算。

# 6 上下文感知门控

真实架构中可能包含多个分支（如4 个），此处为了简化演示，假设为2 个分支。真实中$h_1, h_2, K_1, K_2 \in \mathbb{R}^{256}$，门控维度：

$$d = 256, \qquad \sqrt{d} = 16.$$

门控公式：

$$\alpha_i = \sigma\left(\frac{h_i \cdot K_i}{\sqrt{d}}\right) = \sigma\left(\frac{h_i \cdot K_i}{16}\right), \qquad i = 1, 2.$$

其中$h_i \cdot K_i = \sum_{r=1}^{256} h_i[r] K_i[r]$，是对全部256 维求和，不是只对前4 维求和。

以下数值演示只取前4 维计算，用来说明流程；真实计算需对256 维求和。

```
Branch 1 (first 4 dims only):
  h1[0:4] = [0.5, 0.4, 0.6, 0.3]
  K1[0:4] = [0.805, 0.090, 0.755, 0.190]
  partial_dot_1 = 0.5*0.805 + 0.4*0.090 + 0.6*0.755 + 0.3*0.190 = 0.9485
  (Add remaining 252 dims, then divide by sqrt(256)=16)

Branch 2 (first 4 dims only):
  h2[0:4] = [0.4, 0.5, 0.5, 0.4]
  K2[0:4] = [0.683, 0.145, 0.633, 0.165]
  partial_dot_2 = 0.4*0.683 + 0.5*0.145 + 0.5*0.633 + 0.4*0.165 = 0.7282
  (Add remaining 252 dims, then divide by sqrt(256)=16)
```

若仅为了展示后续数值流程，假设完整256 维点积恰好为：

$$h_1 \cdot K_1 = 15.2, \qquad h_2 \cdot K_2 = 11.65,$$

则：

$$\alpha_1 = \sigma(15.2/16) \approx 0.616, \qquad \alpha_2 = \sigma(11.65/16) \approx 0.590.$$

真实中$\alpha_1, \alpha_2$ 由完整256 维点积得到。

门控后的Value：

$$\tilde{V}_1 = \alpha_1 V \in \mathbb{R}^{256}, \qquad \tilde{V}_2 = \alpha_2 V \in \mathbb{R}^{256}.$$

前4 维示意：

```
V1_tilde[0:4] = 0.616 * [0.740, 0.115, 0.720, 0.185]
              = [0.456, 0.071, 0.444, 0.114]
V2_tilde[0:4] = 0.590 * [0.740, 0.115, 0.720, 0.185]
              = [0.437, 0.068, 0.425, 0.109]
```

两个分支共享同一个256 维$V$，只是门控系数不同。

# 7 后处理：Conv1D + SiLU

真实计算：

$$Y = \mathrm{SiLU}\big(\mathrm{Conv1D}(\mathrm{RMSNorm}(\tilde{V}))\big) + \tilde{V},$$

其中$\tilde{V} \in \mathbb{R}^{256}$，$Y \in \mathbb{R}^{256}$。SiLU 逐元素作用：

$$\mathrm{SiLU}(x) = x \cdot \sigma(x).$$

微缩示例假设RMSNorm 后数值不变，且Conv1D 初始化为恒等映射，因此直接对$\tilde{V}$ 逐元素应用SiLU。以下仅展示前4 维：

```
Branch 1:
  SiLU(0.456)~=0.279, SiLU(0.071)~=0.037, SiLU(0.444)~=0.270, SiLU(0.114)~=0.060
  Y1[0:4] = [0.456+0.279, 0.071+0.037, 0.444+0.270, 0.114+0.060]
          = [0.735, 0.108, 0.714, 0.174]

Branch 2:
  SiLU(0.437)~=0.265, SiLU(0.068)~=0.035, SiLU(0.425)~=0.257, SiLU(0.109)~=0.057
  Y2[0:4] = [0.437+0.265, 0.068+0.035, 0.425+0.257, 0.109+0.057]
          = [0.702, 0.103, 0.682, 0.166]
```

真实中$Y_1, Y_2$ 都是256 维。

# 8 残差注入

真实中：

$$h_1' = h_1 + Y_1 \in \mathbb{R}^{256}, \qquad h_2' = h_2 + Y_2 \in \mathbb{R}^{256}.$$

前4 维示意：

```
h1'[0:4] = [0.5+0.735, 0.4+0.108, 0.6+0.714, 0.3+0.174]
         = [1.235, 0.508, 1.314, 0.474]
h2'[0:4] = [0.4+0.702, 0.5+0.103, 0.5+0.682, 0.4+0.166]
         = [1.102, 0.603, 1.182, 0.566]
```

# 9 最终预测“Paris”

真实中，两个分支的隐藏状态不是简单平均，而是通过拼接后经线性层投影（类似MHA 的$W_O$）或通过可学习的权重进行合并：

$$h_{\text{comb}} = W_O[h_1'; h_2'] \in \mathbb{R}^{256},$$

其中$h_1', h_2' \in \mathbb{R}^{256}$，$W_O \in \mathbb{R}^{256\times 512}$。等价地，可写成两个分支各自投影后相加：

$$h_{\text{comb}} = W_{O1}h_1' + W_{O2}h_2',$$

其中$W_{O1}, W_{O2} \in \mathbb{R}^{256\times 256}$ 是可学习的权重矩阵。

微缩示例用非均匀的对角权重矩阵模拟这一投影过程。假设前4 维的权重为：

$$W_{O1} = \text{diag}(0.6,\ 0.4,\ 0.5,\ 0.5), \qquad W_{O2} = \text{diag}(0.4,\ 0.6,\ 0.5,\ 0.5).$$

则合并后的前4 维按维度逐项加权（Hadamard 积）：

```
dim 0: 0.6*1.235 + 0.4*1.102 = 1.1818
dim 1: 0.4*0.508 + 0.6*0.603 = 0.5650
dim 2: 0.5*1.314 + 0.5*1.182 = 1.2480
dim 3: 0.5*0.474 + 0.5*0.566 = 0.5200

h_comb[0:4] = [1.1818, 0.5650, 1.2480, 0.5200]
```

真实中$h_{\text{comb}}$ 是256 维。LM Head 中每个候选词的权重也是256 维：

$$w_{\text{paris}}, w_{\text{capital}} \in \mathbb{R}^{256}.$$

logit 为完整256 维点积。前4 维计算示意：

```
h_comb[0:4] = [1.1818, 0.5650, 1.2480, 0.5200]
w_paris[0:4]  = [1.0, 0.5, 1.1, 0.4]
w_capital[0:4] = [0.2, 0.9, 0.3, 0.8]

partial_logit_paris = 1.1818*1.0 + 0.5650*0.5 + 1.2480*1.1 + 0.5200*0.4 = 3.045
partial_logit_capital = 1.1818*0.2 + 0.5650*0.9 + 1.2480*0.3 + 0.5200*0.8 = 1.535
```

真实中需对全部256 维求和。若完整logit 恰好为

$$\text{logit}_{\text{paris}} = 3.045, \qquad \text{logit}_{\text{capital}} = 1.535,$$

则通过Softmax 函数转换为词表上的概率分布。真实词表包含99,092 个词，为方便展示，此处仅以得分最高的两个候选词为例：

$$P(\text{Paris}) = \frac{e^{3.045}}{e^{3.045} + e^{1.535}} \approx 0.819.$$

输出：`Paris`

# 10 真实规模

| 步骤 | 真实V4.1-Flash |
|---|---|
| 压缩词表 | 99,092 |
| 哈希头 | 8 |
| head_dim | 256 |
| 每行字节 | 256 FP8 |
| 每token 每层取行数 | 24 |
| 每token 每层读取 | 6144 字节 |
| Engram 层数 | 2 |
| 每层表行数 | 约3.84 亿 |
| 单层表大小 | 约94.6 GiB |
| 两层总表 | 约189 GiB |
| 推理吞吐损失 | $< 3\%$ |

真实推理时，约189 GiB 的表不在HBM 中，而是放在主机内存或NVMe，通过后台RDMA 预取，与Transformer 计算重叠。每个token 位置在每层请求24 行，两层合计约12.4 KiB 内存流量。

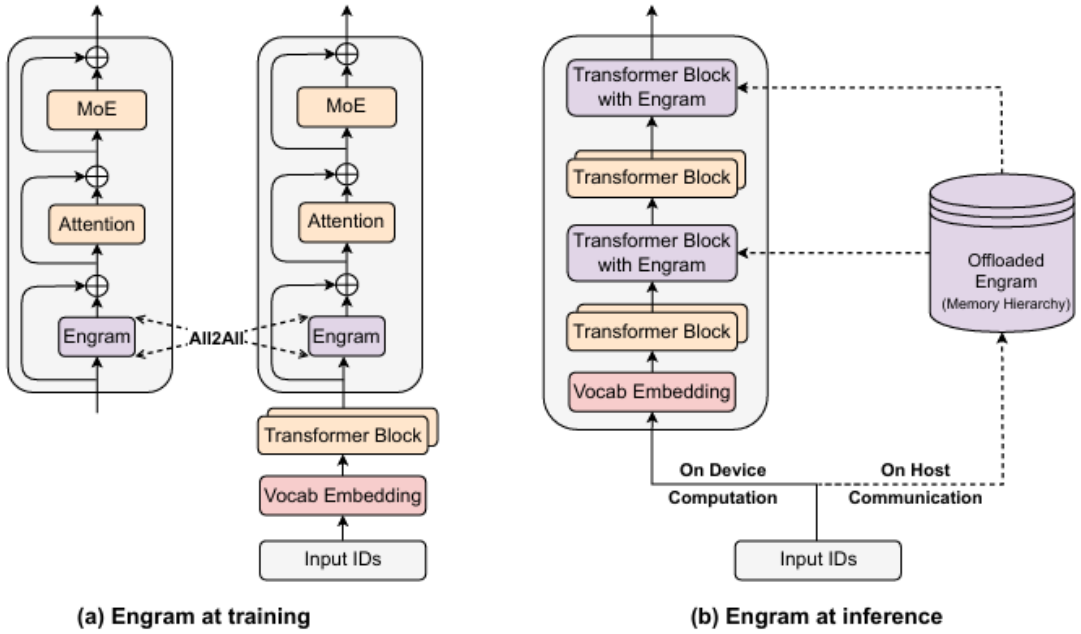



Figure 2: Engram 在训练与推理时的系统架构。（a）训练阶段：海量嵌入表分片存储，通过All2All 通信原语跨设备检索与分发梯度，实现容量的线性扩展；（b）推理阶段：Engram 表存放于主机内存或NVMe 中，通过Offloaded Engram Memory Hierarchy 管理，利用后台RDMA 预取与计算重叠，极大降低HBM 占用。